UNIT – 1 ASSIGNMENT

SOFTWARE TESTING

EVALUATION OF THE QUALITY PROCESS AND DEFECT PROFILE OF THE MEDIQUEUE HOSPITAL APPOINTMENT APPLICATION

STUDENT DETAILS	INFORMATION
Name	AATHISRI
Register Number	192521252
Course Code	CSA3712
Assignment	Unit – 1 Assignment
Case Study	MediQueue – Hospital Outpatient Appointment Application

Submitted in partial fulfilment of the course requirements.

ACADEMIC SUBMISSION

CONTENTS

- Introduction and Case Study Background
- Task 1 – Evaluate Testing as an Engineering Activity
- Task 2 – Evaluate the Role of Process in Software Quality
- Task 3 – Evaluate Testing as a Process
- Task 4 – Apply and Critique Basic Definitions
- Task 5 – Evaluate the Application of Testing Principles
- Task 6 – Evaluate the Tester’s Role
- Task 7 – Trace the Origins of Defects
- Task 8 – Evaluate Defect Classes and Severity
- Task 9 – Design and Appraise a Defect Repository
- Task 10 – Evaluate Defect Examples and Prevention Strategies
- Task 11 – Recommend and Defend a Quality Improvement Plan
- Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Quality Process
- Conclusion
- References

INTRODUCTION

Software testing is a systematic activity used to evaluate whether a software application satisfies specified requirements and behaves reliably under expected and unexpected conditions. It helps identify defects before they affect users, protects data, improves usability and provides evidence for release decisions. Testing is not limited to executing a program after coding; it begins when requirements are discussed and continues through design, development, integration, deployment and maintenance.

MediQueue is a hospital application used by approximately 6,000 patients every day to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. The reported incidents and release-defect figures show that quality was not managed systematically. The central issue was not simply that individual developers made mistakes. Requirements, design, implementation, testing, documentation and release decisions were not controlled as one connected quality process.

Case Study Background

The application supports important hospital activities. A defect in appointment booking may create duplicate tokens or allow bookings when a department is closed. A billing defect may charge an incorrect amount. A patient-data defect may expose one patient's information to another person. A slow schedule screen may delay hospital staff. Therefore, MediQueue must be evaluated for functional correctness as well as privacy, security, performance, availability, usability and data integrity.

Case-study information	Details
Daily usage	Approximately 6,000 patients
Main functions	Appointment booking, consultation-fee payment, token generation and prescription viewing
Development team	Five developers
Reported incidents	Ten incidents, labelled I1 to I10
Detected defects	100 defects in the supplied phase-wise data
Production detection	60 defects, representing 60% of all detected defects
Main quality concern	Late detection, weak reviews, informal testing and insufficient release controls

This assignment evaluates the original process, classifies the incidents, estimates the cost of late defect detection and proposes a practical improvement approach suitable for a five-developer team. Calculations use the supplied figures. Where the exact detection phase of an escaped defect is not identified, assumptions are stated clearly.

Objectives of the Assignment

- To explain why software testing must be treated as an engineering activity.
- To understand how a defined development and testing process improves product quality.
- To identify suitable testing activities and phases for each reported incident.
- To distinguish error, fault, failure, test case, test oracle and test bed.
- To classify defects by origin, type, severity and priority.
- To propose practical prevention, tracking and continuous-improvement methods.

TASK 1 – EVALUATE TESTING AS AN ENGINEERING ACTIVITY (5 MARKS)

ADDITIONAL ELABORATION – Why testing must be planned: A planned testing activity connects each requirement to a test condition and a measurable result. For example, the requirement that every appointment

must receive a unique token should be linked to test cases for ordinary bookings, two users booking at the same time, repeated requests and system recovery after a network interruption. Without this connection, a team may test the normal path while missing the conditions that create serious failures.

ADDITIONAL ELABORATION – Quality evidence and exit criteria: Testing should produce evidence that can be reviewed by another team member. Useful evidence includes the test case identifier, software build, environment, input data, actual result, pass/fail status and defect reference. Before release, the team should agree on exit criteria such as completion of all critical tests, no unresolved critical defects, successful regression testing and documented approval of remaining risks. These criteria make release decisions less dependent on personal opinion.

Testing in MediQueue cannot reasonably be called a complete engineering activity. Engineering requires a planned, measurable, repeatable and standards-based method. The team mainly relied on developers checking their own code near release and a management demonstration. A demonstration may reveal visible mistakes, but it cannot provide sufficient evidence that the application is fit for clinical and operational use.

Characteristics of Engineering-Based Testing

Characteristic	Evidence in MediQueue	Effect and required practice
Planning	No separate tester, documented strategy, risk analysis, schedule or coverage targets.	Important paths such as simultaneous bookings, refunds, age boundaries and PDF generation were not deliberately planned. Prepare a risk-based test plan before implementation.
Measurement	Release approval depended on a demo; no stated coverage, pass-rate, escaped-defect or performance targets.	A demo cannot measure concurrency, security, correctness or load. Track requirement coverage, defect leakage, severity, response time and regression results.
Standards	No formal requirement/design reviews or stated coding, test-data, security or documentation standards.	Undocumented business rules and inconsistent instructions were not challenged. Adopt templates, review checklists and secure coding standards.
Repeatability	Testing was informal and performed by developers just before release; controlled test cases and data were not described.	The refund test passed against an incorrect expected result (I8). Use version-controlled test cases, controlled data and repeatable automated suites.
Independence	Developers tested their own code; no independent quality role existed.	Shared assumptions can allow defects to pass. An independent tester should challenge requirements and verify high-risk behaviour.

Incident Evidence

The ten incidents demonstrate that different engineering checks were required. I1 required concurrency testing to detect duplicate token allocation. I2 required confirmation of the Sunday closure rule with hospital staff. I3 required an approved tax rule and fee reconciliation. I4 required input-boundary testing. I5 required an age-eligibility boundary test. I6 required patient-data isolation and concurrent PDF tests. I7 required a documentation review. I8 required independent verification of the expected refund result. I9 required integration and configuration testing after an API-key change. I10 required performance measurement using representative hospital Wi-Fi.

Recommended Engineering Testing Workflow

1. Collect and review stakeholder requirements, including clinical and operational rules.

2. Define measurable acceptance criteria and identify high-risk functions.
3. Review architecture, data flow, security controls and external dependencies.
4. Create traceable test cases, expected results, test data and test conditions.
5. Prepare a controlled test environment that represents real devices, networks and services.
6. Execute unit, integration, system, acceptance and non-functional tests.
7. Record defects with evidence, severity, priority, ownership and reproduction steps.
8. Retest corrections, run regression tests and evaluate remaining risks.
9. Release only when agreed exit criteria and business approvals are satisfied.

Therefore, the team expected quality to emerge from coding and a successful demonstration. A proper engineering approach builds quality into every stage and records objective evidence that the application is ready for use.

TASK 2 – EVALUATE THE ROLE OF PROCESS IN SOFTWARE QUALITY (2 MARKS)

ADDITIONAL ELABORATION – People, process and product: People provide knowledge and technical skill, the process organizes their work, and the product is the result. If the process is weak, important activities may be skipped even when the team members are capable. A defined process does not remove the need for skill; it makes good practice repeatable and helps new team members understand what is expected.

ADDITIONAL ELABORATION – Continuous process improvement: The team should review defect trends after every release. If many defects originate in requirements, it should improve stakeholder interviews and requirement reviews. If defects repeatedly occur during integration, it should strengthen interface specifications and integration tests. Improvement should be based on evidence rather than simply adding more testing activities.

Software quality is produced by capable people working within a reliable process. Skilled developers are important, but individual skill cannot compensate for missing requirements, reviews, independent verification, controlled test data, release criteria and feedback. A good process guides the team to perform the correct activity at the correct time and ensures that important decisions are documented.

For example, a more experienced developer might have avoided a coding mistake, but would not automatically discover that the Sunday closure rule was never specified (I2). Similarly, the developer may implement the GST calculation exactly as understood, while the hospital's actual policy remains misunderstood (I3). A documentation defect such as a menu that does not exist (I7) also requires a document-to-product review, not only programming skill.

Evidence of Process Weakness

The supplied data shows that 60 out of 100 defects were detected only after production release. Requirements and design had no formal reviews, while coding involved only developer self-checks. On a simplified Testing Maturity Model (TMM) view, the team is close to Level 1 – Initial, where testing is ad hoc, reactive and dependent on individuals. A repeatable, managed process has not yet been established.

How Process Improves Quality

- Consistency: common procedures ensure that different developers follow the same quality practices.
- Early defect prevention: requirement and design reviews identify misunderstandings before code is written.
- Accountability: responsibilities, defect owners and approval points are clearly assigned.
- Traceability: requirements can be linked to test cases, results and defects.

- Measurement: defect trends and test results help managers make evidence-based decisions.
- Continuous improvement: root-cause analysis prevents the same type of defect from recurring.

The hospital should first establish defined and measured practices and gradually move toward managed testing. Recruiting more developers alone is not a complete solution.

TASK 3 – EVALUATE TESTING AS A PROCESS (6 MARKS)

ADDITIONAL ELABORATION – Test levels and their purpose: Unit testing checks a small component, such as the function that calculates a consultation fee. Integration testing checks whether connected components work together, such as the appointment service and payment gateway. System testing evaluates the complete MediQueue application against its requirements. Acceptance testing allows hospital representatives to confirm that the workflows support actual outpatient operations. Each level finds different kinds of problems, so one level cannot replace all the others.

ADDITIONAL ELABORATION – Test techniques suitable for MediQueue: Equivalence partitioning divides input values into groups expected to behave similarly, such as valid and invalid patient names. Boundary-value analysis focuses on limits, such as a maximum name length or the age threshold for senior priority. Decision-table testing is useful when booking eligibility depends on several conditions, such as clinic day, doctor availability and patient category. Exploratory testing allows a tester to investigate realistic user journeys and unusual combinations that may not be covered by scripted cases.

ADDITIONAL ELABORATION – Why regression testing is essential: A correction to one module can unintentionally affect another module. For example, changing appointment eligibility may affect token generation, billing or patient notifications. Regression testing reruns selected earlier tests after a change to confirm that existing behaviour remains correct. The most important regression tests should be automated and executed whenever relevant code is changed.

Testing should operate throughout the Software Development Life Cycle (SDLC). It is a continuous process containing connected activities. Each stage produces information needed by the next stage, and early feedback reduces the cost and risk of correcting defects.

Main Activities in the Testing Process

Activity	Detailed explanation and MediQueue example
1. Test planning	Define scope, objectives, risks, resources, schedule, responsibilities, test levels, tools and release exit criteria. For MediQueue, patient privacy and token allocation should receive high risk priority.
2. Test analysis and design	Review requirements, identify test conditions, equivalence classes and boundary values, create test cases, expected results, test data and traceability. For example, test patient age around the 60-year eligibility boundary.
3. Environment and data preparation	Configure realistic devices, hospital Wi-Fi, payment and SMS sandboxes, concurrent users and privacy-safe patient records. Avoid using real patient data in ordinary test environments.
4. Test execution	Perform reviews, static checks, unit, integration, system, acceptance, performance, security and regression testing. Record actual results and attach evidence.
5. Evaluation and reporting	Compare actual results with an independently approved expected result, assess coverage and unresolved risks, retest fixes and prepare a test summary.
6. Test closure	Confirm completion criteria, document residual risks, archive test artefacts, report metrics and record lessons for the next release.

Verification and Validation

Verification asks, “Are we building the product correctly?” It checks whether each work product follows its specification. Examples include reviewing requirements, inspecting design diagrams, checking code standards and confirming that a calculation follows the approved rule. Validation asks, “Are we building the correct product?” It checks whether the complete application meets the real needs of patients and hospital staff. User acceptance testing and realistic workflow testing are important validation activities.

Best Detection Phase for Each Incident

Incident	Best phase(s)	Reason and suitable check
I1	Design review and integration/system test	Review atomic token allocation; send simultaneous booking requests for the same doctor and verify unique token numbers.
I2	Requirements review and acceptance test	Confirm Sunday closure with outpatient staff and test that booking is rejected on a closed day.
I3	Requirements review and unit/integration test	Approve the GST rule and reconcile the displayed fee, calculated amount and payment charge.
I4	Coding/unit test	Test names at 40, 41 and longer characters; verify graceful validation instead of an application crash.
I5	Requirements review and unit/system test	Define age calculation on the appointment date and test the 60th birthday boundary.
I6	Design review and integration/system test	Review transaction and session isolation; generate PDFs concurrently and verify that each contains only the correct patient’s data.
I7	Documentation review and usability/acceptance test	Check every manual instruction and menu name against the released interface.
I8	Test design review and execution evaluation	Independently verify the expected refund result and require peer review of financial test oracles.
I9	Integration/configuration test	Test API-key rotation, authentication failure handling and the SMS provider contract.
I10	Performance test	Measure schedule-screen response time on representative hospital Wi-Fi with realistic data and concurrent use.

Cost of Late Defect Detection

The following calculation applies the supplied cost per defect to the phase in which each defect was detected. The detected counts total 100.

Detection phase	Defects detected	Cost per defect	Calculated cost
Requirements	2	₹500	₹1,000
Design	3	₹1,500	₹4,500
Coding	15	₹3,000	₹45,000
Testing	20	₹8,000	₹1,60,000
Production	60	₹40,000	₹24,00,000
TOTAL	100	—	₹26,10,500

Production defects alone cost 60 × ₹40,000 = ₹24,00,000, approximately 91.9% of the total calculated cost. The average cost per detected defect is ₹26,10,500 ÷ 100 = ₹26,105. This demonstrates the financial impact of allowing defects to escape into production.

Illustrative Early-Review Saving

The case information records 30 requirements defects and 22 design defects, while only 2 and 3 respectively were detected in those phases. Therefore, 28 requirements defects and 19 design defects were not detected at their introduction phase. If formal reviews had found all 47 missed defects at their respective phase costs, and if all 47 would otherwise have been found in production, the assumed production cost would be 47 × ₹40,000 = ₹18,80,000.

The assumed early-detection cost would be (28 × ₹500) + (19 × ₹1,500) = ₹14,000 + ₹28,500 = ₹42,500. Estimated saving = ₹18,80,000 − ₹42,500 = ₹18,37,500.

This is a scenario estimate, not a guaranteed saving. The data does not identify the eventual detection phase of each of the 47 defects. Some may have been detected during coding or testing. The calculation also assumes reviews find all 47 defects and excludes the cost of conducting reviews. The actual saving may be lower. Nevertheless, the figures strongly support earlier reviews and testing.

TASK 4 – APPLY AND CRITIQUE BASIC DEFINITIONS (4 MARKS)

ADDITIONAL ELABORATION – Relationship between the terms: An error is made by a person; that error may introduce a fault into a requirement, design, program or document. When the faulty software is executed under a particular condition, it may produce a failure. A test case is used to expose the behaviour, while the test oracle determines what the correct result should be. The test bed provides the environment in which the test is performed. Understanding this chain helps the team investigate the actual cause instead of applying the general label “bug” to every issue.

ADDITIONAL ELABORATION – Importance of a reliable test oracle: A test oracle must come from an authoritative source. For MediQueue, fee and refund expectations should be approved by the hospital’s finance or administration team, while clinical eligibility rules should be confirmed by the responsible hospital department. If a tester invents an expected value or copies an outdated rule, the test may report a false Pass or false Fail. Therefore, expected results need ownership, review and version control.

A shared testing vocabulary helps developers, testers and managers describe quality problems accurately. The terms below are related but do not mean the same thing.

Term	Meaning	MediQueue example
Error	A human mistake made during requirements, design, coding or testing.	An analyst misunderstands the hospital’s GST rule or fails to ask whether Sunday clinics operate.
Fault / defect	An incorrect requirement, design, code or document created because of an error.	An incorrect tax calculation or unsafe shared state in PDF generation.
Failure	Observable incorrect behaviour when the software is executed.	A patient sees a fee of ₹500 but is charged ₹550.
Test case	A documented set of preconditions, inputs, actions and expected results.	Attempt to book an appointment on Sunday and expect the system to reject it.
Test oracle	A trusted source used to decide the correct expected result.	An approved hospital fee policy or refund rule.
Test bed	The configured hardware, software, network,	The MediQueue staging system connected to

	services and data used to run tests.	payment and SMS sandboxes.

Examples Linking Error, Defect and Failure

Incident	Likely human error	Fault / defect	Observed failure
I3	GST rule misunderstood by analyst/developer.	Incorrect tax or fee calculation, or inconsistent pricing logic.	Patient sees ₹500 but is charged ₹550.
I5	Age eligibility interpreted using the current date instead of the appointment date.	Incorrect senior-citizen eligibility condition.	A patient turning 60 on the appointment day receives no priority.
I6	Concurrency and patient-data isolation risk overlooked.	Shared mutable state or missing transaction/session isolation in PDF generation.	A prescription contains another patient's details.

I8 demonstrates why terminology matters. The test case was marked Pass because its expected result was wrong. The execution may have matched the expected result, but the oracle itself was invalid. Calling every problem a "bug" hides whether the cause was a human error, a product fault, an execution failure or a bad test artefact.

Recommended Shared Defect Vocabulary

- Define defect states such as New, Assigned, In Progress, Fixed, Retest, Reopened and Closed.
- Define severity and priority separately so that impact and urgency are not confused.
- Record the root-cause category, detection phase and affected module.
- Specify mandatory test-case fields and who approves expected results.
- Require evidence and privacy-safe reproduction steps for every reported defect.

TASK 5 – EVALUATE THE APPLICATION OF TESTING PRINCIPLES (3 MARKS)

ADDITIONAL ELABORATION – Using principles to select test coverage: Because exhaustive testing is impossible, the team should select tests according to risk, usage frequency and possible impact. Frequently used functions such as appointment booking and payment deserve broad coverage. Functions that process sensitive patient information need strong privacy and authorization checks even if they are used less often. Boundary cases, invalid inputs and recovery conditions should be included alongside normal successful workflows.

ADDITIONAL ELABORATION – Reviewing the test suite over time: The pesticide paradox means that a test suite that never changes may stop finding new defects. The team should add tests whenever a new defect is discovered, review tests when requirements change and remove obsolete cases. This keeps the suite relevant as the application and its external services evolve.

Testing principles guide the team in selecting effective tests and understanding the limits of testing. Applying these principles to MediQueue helps explain why the earlier testing approach was insufficient.

Testing principle	Meaning and application to MediQueue
Testing shows the presence of defects, not their absence.	A clean management demo did not prove correctness. I1, I6 and I10 could remain hidden until concurrent or realistic use.
Exhaustive testing is impossible.	There are many users, dates, names, doctors, payment values and timing combinations. Risk-based representative and boundary tests are necessary.
Early testing saves time and cost.	Requirements and design reviews were absent, while 60 defects reached production at ₹40,000 each.

Defects cluster together.	Weak requirements and incidents involving booking, billing and patient data suggest that several faults may exist in the same weak modules. Focus testing on high-risk areas.
Tests wear out (pesticide paradox).	Repeating the same developer demonstration may find familiar issues but miss new API-key, concurrency and boundary conditions. Review and expand test cases.
Testing is context-dependent.	A hospital system needs strong privacy, correctness, availability and patient-safety checks compared with a low-risk personal application.
Absence-of-errors fallacy.	Even if there were no coding faults, I2 and I7 would still make the application unsuitable because it would not meet actual hospital needs.

The principle with the largest likely effect is early testing. Formal requirements and design reviews could have challenged the missing Sunday rule, GST interpretation, age policy and concurrency design before implementation. Given the ₹24 lakh production detection cost, preventing or detecting defects earlier offers the greatest combined safety and financial benefit.

TASK 6 – EVALUATE THE TESTER'S ROLE (3 MARKS)

ADDITIONAL ELABORATION – Tester independence and collaboration: Independence does not mean that testers work separately from developers. Testers and developers should collaborate during requirement discussions and design reviews, while preserving an objective approach to checking results. Developers are best placed to test individual code units, whereas an independent tester can challenge assumptions and examine the complete user journey.

ADDITIONAL ELABORATION – Communication and defect reporting: A useful defect report should be clear, factual and reproducible. It should state the affected build, preconditions, exact steps, expected result, actual result and supporting evidence. The tester should avoid blaming individuals and should explain the impact on patients, hospital staff, data or business operations. This helps the team prioritize and resolve the issue efficiently.

An independent tester would probably have reduced the number of defects reaching patients, but would not automatically prevent every defect. The tester provides an objective view of the product and checks whether the application behaves as users and stakeholders expect.

Major Responsibilities of a Software Tester

- Review requirements and identify missing, ambiguous or conflicting rules.
- Identify product risks and decide which functions need the greatest testing effort.
- Design test scenarios, test cases, test data and expected results.
- Execute tests and record clear evidence of actual behaviour.
- Report defects objectively with reproduction steps, severity and priority.
- Verify fixes and perform regression testing to ensure existing features still work.
- Communicate test coverage, unresolved risks and release recommendations.
- Protect confidential information by using approved, privacy-safe test data.

How an Independent Tester Could Help

A tester could most likely have detected I4 through boundary testing, I8 through test-oracle review, I3 through fee reconciliation, I7 through manual and interface checks, I9 through API-key rotation tests and I10 through performance testing. I1 and I6 require deliberate concurrency tests and design discussions. I2 and I5 require clarification of requirements with hospital staff.

Skills Required for a Healthcare Software Tester

- Curiosity and analytical thinking to identify unusual user conditions.
- Independence and clear communication with developers, hospital staff and managers.
- Attention to detail when validating fees, dates, patient identity and records.
- Knowledge of functional, integration, regression, performance and security testing.
- Awareness of privacy, access control, auditability and reliability.
- Ability to document defects and explain risk without exaggerating or hiding evidence.

For a five-developer team, a large separate testing department may not be practical. A better model is to appoint a dedicated quality owner/tester or rotate a trained tester role with protected time, while developers remain responsible for unit tests.

TASK 7 – TRACE THE ORIGINS OF DEFECTS (4 MARKS)

ADDITIONAL ELABORATION – Root cause versus symptom: The visible failure is not always the true origin of a defect. If a patient receives a duplicate token, changing the screen message may hide the symptom but will not correct unsafe token allocation. Root-cause analysis should examine the requirement, design decision, code, test coverage and release controls that contributed to the problem. Corrective actions should address the earliest controllable cause.

ADDITIONAL ELABORATION – Practical root-cause review: For every critical or repeated defect, the team can ask: What happened? Under which conditions? Which requirement or design decision was involved? Why did earlier tests not detect it? What change will prevent recurrence? The answers should be recorded in the defect repository and converted into a new review checklist, automated test or process improvement.

Defect origin refers to the lifecycle stage or work product where a problem was first introduced. Identifying the origin is important because fixing only the visible symptom may allow the same underlying issue to occur again.

Incident	Primary origin	Reasoned explanation
I1	Design / coding	Token allocation was not made atomic or uniquely constrained for concurrent requests. The simultaneous-booking condition should have been addressed in design and verified in integration tests.
I2	Requirements	The Sunday closure rule was never written. This is mainly a requirements elicitation and approval failure, not merely a coding error.
I3	Requirements / coding	The GST rule was misunderstood, indicating unclear or incorrectly approved business rules; calculation implementation and display/charge consistency then failed.
I4	Coding	A name-length boundary causes a crash, pointing to inadequate input validation or unsafe handling. A requirement may also have omitted the maximum length.
I5	Requirements / coding	The eligibility rule for age on the appointment date was not made precise or was implemented with the wrong date comparison.
I6	Design / coding	Concurrent PDF generation exposed inadequate patient-data isolation, shared state or transaction handling.
I7	Documentation	The manual describes a menu that does not

		exist, showing that documentation was not validated against the product.
I8	Testing	The test passed using an incorrect expected value, showing weak test-oracle creation and review.
I9	Environment / integration	A changed external provider API key was not handled or tested; configuration management and integration assumptions were weak.
I10	Design / coding	Slow schedule loading suggests inefficient query, data retrieval or screen design; performance requirements and representative load testing were missing.

Root-Cause Analysis and Prevention

The pattern is systemic: requirements and design weaknesses are prominent, testing was late and insufficient, and production detection was expensive. A root-cause review should ask why the defect was introduced, why earlier checks did not detect it, what process condition allowed it and what preventive action will stop recurrence.

- Use stakeholder interviews and signed business-rule reviews to prevent missing requirements.
- Use architecture and design inspections for concurrency, data isolation and performance.
- Apply coding standards, peer reviews, input validation and automated unit tests.
- Independently review financial and eligibility test oracles.
- Validate user manuals against the released application.
- Maintain configuration records and integration tests for external services.

Some incidents have multiple origins. The table assigns the earliest actionable cause as primary and notes implementation or testing contributions where relevant.

TASK 8 – EVALUATE DEFECT CLASSES AND SEVERITY (4 MARKS)

ADDITIONAL ELABORATION – Defect triage and prioritization: During triage, the team should consider impact, likelihood, number of affected users, data sensitivity, frequency, available workaround and release timing. A defect with high severity may need immediate containment even if a permanent fix takes longer. A lower-severity defect may still receive high priority when it blocks a release or affects a time-critical workflow.

ADDITIONAL ELABORATION – Examples of severity and priority decisions: A patient-data mix-up is critical because it affects confidentiality and may create clinical risk; it should be contained immediately. A spelling mistake in a non-critical help page is usually low severity and can be scheduled normally. A broken cancellation instruction may appear less severe, but its priority increases if patients need to cancel before a deadline. The team should document the reason for each priority decision.

Defect classification groups problems according to the work product or technical area in which they occur. Severity measures the impact if the defect occurs. Priority measures how urgently the team should fix it, considering exposure, frequency, available workarounds and release plans.

Common Defect Classes

- Requirements/specification defects: missing, ambiguous or incorrect business rules.

- Design defects: problems in architecture, data flow, concurrency, security or performance design.
- Coding defects: incorrect logic, boundary handling, validation or exception handling.
- Testing defects: incorrect test cases, test data, expected results or test oracles.
- Documentation defects: inaccurate user instructions, help text or operational procedures.
- Integration/environment defects: incorrect configuration or incompatibility with an external service.

ID	Defect class	Severity	Priority	Justification
I1	Design/coding – concurrency	Critical	P1 – Immediate	Duplicate tokens can disrupt patient order and clinical operations; simultaneous requests expose a race condition.
I2	Requirements – missing business rule	High	P1 – Immediate	Patients can book when the department is closed, causing wasted visits and operational confusion.
I3	Requirements/coding – financial rule	High	P1 – Immediate	Incorrect charges damage trust and may create financial or compliance complaints.
I4	Coding – input boundary/robustness	Medium	P2 – High	A long name crashes the app; the issue blocks affected users but is limited to certain inputs.
I5	Requirements/coding – date/eligibility logic	High	P1 – High	An eligible senior patient may lose priority, potentially affecting access to care.
I6	Design/coding – data isolation/privacy	Critical	P1 – Immediate	One patient's prescription may expose another patient's health information and create clinical risk.
I7	Documentation – inconsistency	Low	P2 – Normal	Cancellation instructions are wrong, but a workaround may exist. Priority rises if cancellation is time-critical.
I8	Testing – incorrect oracle	High	P1 – High	A false Pass undermines confidence in test results and may allow incorrect refund processing.
I9	Integration/environment configuration	High	P1 – High	SMS reminders fail after key rotation; patients may miss appointments, although core booking may still work.
I10	Design/coding – performance	Medium	P2 – High	A 25-second schedule load delays staff workflow and may become severe during peak clinic periods.

Severity Versus Priority

Severity and priority are not identical. Severity describes the seriousness of the impact, while priority describes the order and urgency of correction. I1 is critical because duplicate tokens can affect patient flow, so it deserves immediate priority. I7 may have low direct harm and a workaround, so its severity is lower; however, if cancellation is needed to free a slot or avoid a fee, its priority can rise.

The first defect classes to improve are requirements/specification defects, followed closely by design defects. They affect many downstream components and are usually cheaper to correct before code exists. Data privacy and concurrency defects such as I6 and I1 still require immediate containment regardless of the longer-term improvement order.

TASK 9 – DESIGN AND APPRAISE A DEFECT REPOSITORY (3 MARKS)

ADDITIONAL ELABORATION – Repository workflow and governance: A repository is useful only when the team follows a common workflow. Every new report should be reviewed for clarity and duplicates, assigned to an owner, updated when its status changes and closed only after retesting. Access should be limited according to role, and sensitive patient details should be removed from screenshots, logs and sample files.

ADDITIONAL ELABORATION – Using repository data for management: The repository should support periodic reports showing the modules with the most defects, the stages where defects are detected, the number of reopened issues and the average time to repair. These reports help managers decide whether to improve requirements, design reviews, coding practices or test coverage. The purpose is not to compare or blame individual developers, but to improve the process.

MediQueue should use one controlled defect repository, such as a project issue tracker, rather than scattered e-mails and a loosely maintained spreadsheet. A repository provides a central record of defects, supports team communication, tracks ownership and helps management identify repeated quality problems.

Essential Fields in a Defect Record

Field	Purpose
Unique defect ID, title and module	Provides a reliable reference and supports trend analysis.
Affected version/build	Identifies the exact software version in which the issue occurred.
Reporter, date found and detection phase	Shows where defects are discovered and helps measure defect leakage.
Environment/device/network	Helps reproduce environment-specific failures.
Preconditions and reproduction steps	Explains how to repeat the problem.
Input/test data and expected versus actual result	Makes the defect clear and actionable.
Evidence: logs, screenshots, request IDs and sanitized files	Supports diagnosis without exposing real patient information.
Defect class, origin/root cause, severity and priority	Enables risk-based triage and prevention planning.
Status, owner, target fix version and code change	Establishes accountability and traceability.
Retest result, regression test and closure date	Proves the fix works and reduces recurrence.
Links to requirement, design item, test case and release	Connects defects to the software lifecycle.

Defect Life Cycle

A typical defect life cycle is: New → Reviewed/Triaged → Assigned → In Progress → Fixed → Ready for Retest → Retested → Closed. If the issue remains, it is Reopened and returned to the responsible owner. A

duplicate or invalid report should be marked accordingly with a reason. The team should agree on these states so that reports are consistent.

Benefits and Limitations

The repository can generate trends such as defects by origin, module, severity and detection phase, escaped-defect rate, reopen rate and average repair time. Root-cause reviews can turn recurring issues into prevention actions. E-mails and spreadsheets are weaker because they permit duplicate reports, inconsistent labels, missing evidence, poor version links, unclear ownership and limited audit history. Access controls and sanitization are essential because defect reports may otherwise contain patient information.

TASK 10 – EVALUATE DEFECT EXAMPLES AND PREVENTION STRATEGIES (4 MARKS)

ADDITIONAL ELABORATION – Prevention versus correction: Correction removes a defect that already exists, while prevention changes the way work is performed so that similar defects are less likely to occur. For example, correcting the Sunday booking condition fixes the current behaviour; adding a signed business-rule checklist helps prevent other missing operational rules. Both are necessary, but prevention reduces repeated rework and late release costs.

ADDITIONAL ELABORATION – Checking whether prevention worked: The team should measure whether the selected action reduces recurrence. After introducing business-rule reviews, it can track missing-requirement defects. After adding concurrency tests, it can track duplicate-token incidents and related defects. If the measure does not improve, the team should review whether the action was followed and whether it addressed the true root cause.

Defect prevention is more effective than repeatedly correcting the same type of failure after release. A prevention strategy should address the process condition that allowed the defect to occur, rather than only correcting the immediate code.

Selected incident/class	Root cause	High-value process change	Effort versus benefit
I2 – Requirements defect	Sunday closure policy was never elicited or documented.	Require a signed business-rule review with outpatient operations before a requirement is approved.	Low effort: a short workshop and checklist. High benefit: prevents invalid bookings and expensive downstream changes.
I1 – Design/concurrency defect	Token generation did not safely handle simultaneous requests.	Make concurrency and data-integrity design inspection mandatory for shared booking resources.	Moderate effort: architecture review and focused concurrent testing. High benefit: prevents duplicate tokens and protects patient flow.
I8 – Testing defect	The expected refund result was wrong and was not independently checked.	Require an independently sourced and reviewed test oracle for financial test cases.	Low-to-moderate effort: finance-approved examples and peer review. High benefit: prevents false Pass results and financial errors.

Evaluation of the Strategies

The best value is the requirements review for I2 because it is inexpensive, involves the people who own the operational rule and can prevent a defect before design and coding costs accumulate. However, I1 needs immediate technical containment and I8 requires an oracle control before the next release. The cost data supports this choice: production detection costs ₹40,000 per defect compared with ₹500 at requirements and

₹1,500 at design. Prevention effort is therefore small relative to the potential cost of late correction, although exact savings depend on review effectiveness.

Additional Preventive Controls

- Use requirement templates that include business rules, exceptions, boundaries and user roles.
- Conduct peer reviews for high-risk design and code changes.
- Automate unit, integration and regression tests in the build pipeline.
- Use approved test data and independently reviewed expected results.
- Perform periodic security, privacy and performance reviews.
- Conduct root-cause analysis for critical and repeated defects.

TASK 11 – RECOMMEND AND DEFEND A QUALITY IMPROVEMENT PLAN (4 MARKS)

ADDITIONAL ELABORATION – Implementation in manageable steps: The team should introduce the plan gradually. First, address critical patient-flow and privacy issues. Next, standardize requirement reviews, defect reports and release checklists. Then automate stable unit, API and regression tests. Finally, use the collected metrics to decide which practices need further improvement. A gradual approach reduces disruption and makes the plan realistic for a small team.

ADDITIONAL ELABORATION – Release readiness and risk acceptance: A release decision should consider test results and unresolved risks together. The team should not treat a high pass percentage as sufficient if a critical privacy or payment defect remains open. Any accepted residual risk should have a clear explanation, an accountable approver, a mitigation or workaround and a plan for follow-up. Critical patient-safety and privacy risks should not be casually accepted.

I recommend a staged, lightweight quality process rather than a heavyweight process that a five-developer team cannot sustain. The plan should focus first on patient safety, privacy and the defects that have the greatest operational impact.

Phase 1: Immediate Actions Before the Next Release

- Triage and fix I1 and I6 first because of patient-flow and privacy risk.
- Correct I3, I5 and I9 and confirm that their fixes work in integrated workflows.
- Provide clear workarounds and corrected instructions for I7.
- Verify I10 under representative hospital network conditions.
- Review all open critical and high-priority defects before release.

Phase 2: Establish Quality Ownership

Appoint one trained quality engineer or test lead, or rotate the role with protected time. Developers remain responsible for unit tests. The quality owner independently reviews requirements, test design, risk, test evidence and release readiness. This approach creates accountability without requiring a large separate QA department.

Phase 3: Introduce Quality Activities Across the SDLC

Stage	Recommended activity
Before coding	Hold short stakeholder reviews for business rules, boundaries, roles, fees, appointment dates, cancellations and external dependencies. Maintain acceptance criteria and traceability.
During design	Inspect high-risk designs for concurrency, privacy, data isolation, security and performance.

During coding	Use coding standards, peer review, static analysis and automated unit tests in the continuous integration pipeline.
Before every release	Run automated smoke and regression tests, integration tests for payment/SMS, manual exploratory tests for high-risk workflows, security/privacy checks and performance tests.
At release	Use explicit exit criteria: no open critical defects, approved handling of high defects, passing regression suite, reviewed test report, backup/rollback plan and business acceptance.
After release	Monitor logs and service metrics, collect patient/staff feedback, record every defect and conduct root-cause reviews for serious or repeated defects.

Quality Metrics

Metric	What it measures	Why it matters
Defect leakage	Defects found after release compared with total defects found.	Shows whether testing is detecting issues before production.
Requirement coverage	Requirements linked to one or more test cases.	Shows whether important requirements have been tested.
Regression pass rate	Percentage of regression tests that pass.	Indicates whether changes are breaking existing functions.
Defect severity distribution	Number of defects by severity level.	Helps identify safety and operational risks.
Mean time to repair	Average time taken to correct a defect.	Shows the efficiency of defect handling.
Reopen rate	Percentage of defects reopened after a fix.	Can reveal incomplete fixes or weak retesting.

Expected effect: the supplied cost model attributes ₹24,00,000 of ₹26,10,500 to production detection. Moving discovery earlier should reduce escaped defects and average repair cost. The illustrative early-review scenario estimates a possible ₹18,37,500 saving if 47 missed requirements/design defects would otherwise all be found in production; this is an assumption-based scenario, not a forecast.

Trade-offs include time spent reviewing and automating tests, possibly slower feature delivery at first and the risk that one tester becomes a bottleneck. The team can manage these issues by automating repeatable checks, ranking features by risk, pairing developers with the quality owner and keeping reviews short and focused. A five-person team can sustain this process if testing is continuous and proportionate to risk.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE QUALITY PROCESS (8 MARKS)

ADDITIONAL ELABORATION – Importance of API and contract testing: An API contract defines how two systems exchange requests and responses, including fields, formats, authentication and error behaviour. Contract testing checks whether MediQueue and the external health-ID service continue to follow the agreed interface. It can detect incompatible changes before they cause failures in the live system. The team should retain versioned contracts and test against an approved sandbox where available.

ADDITIONAL ELABORATION – Privacy, identity and data integrity controls: The integration must ensure that a record is associated with the correct patient and that only authorized users or systems can access it. Tests should cover valid and invalid identity matches, missing consent or authorization, expired credentials, duplicate messages and interrupted transfers. Synthetic patient data should be used for routine testing, and logs should avoid unnecessary sensitive information.

ADDITIONAL ELABORATION – Monitoring and recovery after deployment: Continuous testing does not end at deployment. The team should monitor integration error rates, response times, authentication failures and delayed or duplicated transactions. It should define how to retry failed requests safely, how to reconcile inconsistent records and when to disable an affected feature. Rollback and incident-response procedures should be tested before they are needed.

A. Impact Analysis (2 Marks)

The release cycle changes from six months to two weeks, and MediQueue now exchanges patient health records with a national digital health ID system maintained by an external agency. These changes affect both the speed of development and the level of risk. The original plan assumed a relatively long release window and mostly hospital-controlled application behaviour. Those assumptions no longer hold.

A two-week release cadence means testing cannot be postponed until the end. Requirements, code, integration and regression checks must run continuously. The tester must participate in backlog refinement, acceptance-criteria review, automation design, continuous integration results and release-risk decisions.

Health-record sharing increases the severity of privacy, identity matching, authorization, data integrity, audit and interoperability defects. A wrong token is operationally serious; a record linked to the wrong patient or disclosed to another hospital can cause lasting clinical and privacy harm.

B. Evaluation of the Original Plan (2 Marks)

The original reviews and test activities remain necessary, but they must be smaller, continuous and automated to fit a two-week cycle. A practical sprint rhythm is to refine and review stories before the sprint, add unit and API tests with each change, run continuous-integration integration and regression tests on every merge, perform focused exploratory/security/performance testing during the sprint and complete a release-readiness review before deployment.

New defect origins include external API contract changes, authentication/token expiry, certificate and key rotation, network outages, rate limits, schema/version incompatibility, incorrect patient identity mapping, duplicate or delayed messages, inconsistent records between hospitals and unclear responsibility for data correction.

The existing defect repository remains a useful foundation but should add external-system name/version, API endpoint/contract version, correlation/request ID, identity-matching impact, data-flow direction, affected data categories, privacy/security classification, dependency owner, external-agency ticket/reference and whether the fault is internal, external or shared.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the original quality process into a continuous, risk-based DevSecOps quality process. A complete redesign is unnecessary because lifecycle reviews, traceability, independent testing and the defect repository remain valid. However, the cadence, automation, security controls and integration strategy must change.

Area	Adapted activity
Backlog and requirements	Define acceptance criteria and API contracts for every health-ID feature. Obtain clinical, privacy and security approval for data-sharing and patient-identity rules.
Design	Review trust boundaries, least-privilege access, encryption in transit and at rest, consent/authorization, audit trails, patient matching, failure recovery and data minimization.
Development and CI	Developers add unit and contract tests. Every pull request runs lint/static analysis, dependency/security scanning, unit tests and API

	contract checks.
Integration and staging	Use a sandbox or certified test environment, synthetic patient data, versioned schemas and simulated failures such as timeout, expired token, invalid key and duplicate response.
Release	Deploy gradually where possible, monitor errors and interoperability metrics, maintain rollback/feature-disable capability and require explicit approval for high-risk health-record changes.
Tester role	Act as a quality advocate across the sprint, review stories and contracts early, maintain automated regression and integration suites, perform exploratory/security/privacy testing and provide evidence-based release recommendations.

D. Evidence-Based Defence (2 Marks)

Modification is the most defensible decision. The original data shows that 60% of defects were detected in production and production detection accounted for ₹24,00,000 of the calculated ₹26,10,500 total. A two-week release cycle makes a final manual testing phase even less realistic; automation and continuous verification are needed to prevent the same late-detection pattern.

The origin analysis identified missing requirements, weak design, coding boundaries, faulty test oracles, documentation gaps and external configuration failures. The new health-ID integration adds dependency and interoperability origins, while shared health records raise the impact of identity, authorization and data-isolation defects to critical. Therefore, the process must add contract testing, security/privacy testing, synthetic-data integration tests and external dependency monitoring.

The five-developer team cannot afford a large separate QA department. A quality owner supported by developer-owned automated tests, peer reviews and a focused independent risk-based test role is practical. The process should be measured after each release using escaped defects, severity, regression pass rate, API contract failures, security findings and time to repair. The hospital should retain the original process foundations but modify them into a continuous DevSecOps process with stronger health-data controls.

Suggested Two-Week Sprint Quality Cycle

Sprint period	Quality activity
Before sprint	Refine requirements, review acceptance criteria, identify risks and agree API contracts.
Days 1–3	Review design, prepare synthetic data and add unit/API tests with implementation.
Days 4–7	Run continuous integration checks, code review, static analysis and contract tests.
Days 8–10	Run integration, regression, exploratory, privacy/security and performance tests.
Days 11–12	Retest fixes, assess defect trends, review unresolved risks and prepare release evidence.
Days 13–14	Obtain business approval, deploy with monitoring/rollback controls and review post-release results.

The exact distribution of work can vary by sprint. The important requirement is that quality activities happen throughout the cycle rather than being compressed into the final days.

CONCLUSION

MediQueue's failures were mainly the result of an immature and reactive quality process, not simply a shortage of talented developers. The evidence—especially 60 production defects and the ₹24 lakh production-fix cost—shows that quality must be built into requirements, design, coding, testing, documentation and release.

A small team can make meaningful progress through clear quality ownership, early stakeholder reviews, independent risk-based testing, automation, controlled test data and a learning-oriented defect repository. The team should measure defect leakage, requirement coverage, regression results, severity trends and repair time so that improvement can be evaluated objectively.

The move to a two-week release cycle and integration with a national digital health ID system increases the need for continuous testing, API contract checks, security and privacy controls, identity validation and interoperability monitoring. The most suitable decision is to retain the useful foundations of the original plan and modify them into a continuous, risk-based DevSecOps quality process.

REFERENCES

10. Course material: Software Testing, Unit I – Testing fundamentals, process, principles and defect management.
11. Case study supplied in the assignment: MediQueue – Hospital Outpatient Appointment Application.
12. International Organization for Standardization. ISO/IEC/IEEE 29119 series – Software Testing (general reference).
13. International Organization for Standardization. ISO/IEC 25010 – Systems and Software Quality Models (general reference).
14. ISTQB Foundation Level syllabus – Software testing fundamentals, test process, test levels and test techniques (general reference).